UNIVERSITY OF WOLVERHAMPTON

BIRATNAGAR INTERNATIONAL COLLEGE

Introductory Programming and Problem-Solving Skills

Week - 7

File Handling

Basanta Singh

Learning Objectives

Navigation

Learning Outcomes

- To understand the purpose and **use of text files.**
- To understand how to **manipulate and process text files in Python**.

Storage Technologies

Advancement of Data Storing

Navigation

Advancement of Data Storing

Storage Technology	Example	Characteristics
Magnetic Storage	Hard drive	Non-volatile
	Magnetic Tape Storage	Non-volatile
Semiconductor Memory	Main memory	Volatile
	USB (Thumb) Drive	Non-volatile
Optical Storage	CD, DVD	Non-volatile

Introduction

Navigation

1. Introduction

- The vast amount of **information** being generated today means that there must also be vast amounts of **storage capacity for all this information.**
- Data has been shared since the early days of computing, initially through **physical storage media** and later through **computer networks**, and now through **cloud computing via the Internet.**

The main question is where do we store data while programming?

UNIVERSITY OF WOLVERHAMPTON

BIRATNAGAR INTERNATIONAL COLLEGE

ing

For Programming We Do Store Data in Various File Format Such As Text, CSV, JSON and so on.

Text Files

What is Text Files?

Navigation

1. What is Text Files?

- **Text File** is a file **containing characters**, **structured as lines of text**.

- Text Files contain the non-printing newline characters, \n. Thus, text files can be directly viewed and created using text editors such as notepad.

- In contrast, binary files can contain various types of data, such as numerical values, and therefore not structured as lines of text. Such files can only be read and written via computer program

Using Text Files

Navigation

1. What is Text Files?

2. Using Text Files

The fundamental operations of all types of file includes

- Opening a file
- Writing to a file
- Closing a file
- Reading from a file

In, further slides, we will see all the operations one by one.

But Before Than That, Let’s Raise Some Question

Questions To Ask!!

Navigation

1. What is Text Files?
2. Using Text Files

Questions to ask!!

- Why and How To Open a Text File?
- Why and How To Read in a Text File?
- Why and How To Write in a Text File?
- How To Close a File, and is it necessary to close?

Open Text Files

Why Open File?

Navigation

1. What is Text Files?
2. Using Text Files
3. Why open File?

- All files must first be opened before they can be read from or written to.
- In Python, when a file is (successfully) opened, a **file object is created** that provides methods for **accessing the file**.
- Further, we will look at **how to open** files for either reading (from) or writing (to) a file.

It's like opening book, no matter we read or write we need to open it first!!

How to Open Text Files?

How To Open Text Files?

Navigation

1. What is Text Files?
2. Using Text Files
3. Why open File?
4. How to open Text files?

- To open a file for reading, the **built-in open function** is used as shown,

```
input_file = open('myfile.txt','r')
```

If the file is successfully opened, a file object is created and assigned to the provided identifier. Here, input_file is identifier.

The first argument is the file name to be opened, ‘myfile.txt’. The second argument indicated file to be in reading format, however this can differ as per the operation.

Let’s Understand Through Example

Navigation

1. What is Text Files?
2. Using Text Files
3. Why open File?
4. How to open Text files?

Example !!

```
def fileOpen():
    input_file = open('BankAcc.txt', 'r')
    print(input_file.readlines())
fileOpen()
```

Here, we can see a file is opened in a read format. The first parameter shows the file and second parameter denotes the read format.

What if filename does not exist?

I/O Error

Navigation

1. What is Text Files?
2. Using Text Files
3. Why open File?
4. How to open Text files?
5. I/O Error

- When opening a file for reading, there are few reasons why an **I/O error may occur**. (We will soon see how to include the ability to catch and handle such errors)

Reasons That May Throw I/O Error

- **Missing File Location**
- **File Name Error**
- **File Type Wrong**

```
>>> open('testfile.txt','r')
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    open('testfile.txt','r')
IOError: [Errno 2] No such file or directory:
'testfile.txt'
```

Note: (uppercase and lowercase letters are treated the same for the file names.)

There won’t be any problem when the text and python file will be on same directory

File Saved In Different Directory?

Navigation

1. What is Text Files?
2. Using Text Files
3. Why open File?
4. How to open Text files?
5. I/O Error
6. File saved in different directory

- When a file is opened, it is **first searched** for in the **same folder/directory** that the program resides in. The programs in the text are written this way. However, an **alternate location can be specified** in the call to open by providing a path to the file:

```
input_file = open('data/myfile.txt','r')
```

- In this case, the file is **searched for the sub directory called data of the directory in which the program is contained**. Thus, its location is relative to the program location.

The Concept of Accessing File From Any Directory!!

Absolute Path

Navigation

1. What is Text Files?
2. Using Text Files
3. Why open File?
4. How to open Text files?
5. I/O Error
6. File saved in different directory
7. Absolute Path

- **Absolute paths** can also be provided giving the location of **file anywhere in the file system**:

```
input_file = open('C:/mypythonfiles/data/myfile.txt','r')
```

How To Read Text Files?

Reading File

Navigation

1. What is Text Files?
2. Using Text Files
3. Why open File?
4. How to open Text files?
5. I/O Error
6. File saved in different directory
7. Absolute Path
8. Reading File

- In previous slide, we discussed that after opening file we can perform operations like **reading and writing**
- There are three **ways/functions** through which we can read files which are **read(), readline(), readlines()**
- All three methods of reading bits of file have different use cases.

Click here https://docs.python.org/3/tutorial/inputoutput.html#methods-of-file-objects

Reading Files Through Iteration

Reading File Through For Loop

Navigation

8. Reading File

8.1 Reading File through while loop

8.2. Reading File through for loop

- It is possible to read the lines of a file by use of the **for statement**:

```
user_input = input("Enter File You Want To Open: ")
input_file = open(user_input, 'r')
   for i in input_file:
       print(i)
```

- Using a for statement, all lines of the file will be read one by one. Using a while loop, however, lines can be read until a given value is found.

Note: Both Loop Have Their Own Importance, as we discussed before.

Let’s Try

Navigation

8. Reading File

8.1 Reading File through while loop

8.2. Reading File through for loop

Tryouts!!

Create a text file named 'myfile.txt'. Enter and execute the following program and observe the results. Make sure that the program and text file reside in the same folder.

```
input_file = open('myfile.txt','r')
for line in input_file:
    print(line)
```

How To Write In Text Files?

Writing Files

Navigation

1. What is Text Files?
2. Using Text Files
3. Why open File?
4. How to open Text files?
5. I/O Error
6. File saved in different directory
7. Absolute Path
8. Reading File
9. Writing File

- **To open a file for writing, the built-in write function is used as shown,**

```
output_file = open('mynewfile.txt','w')
```

If the file is successfully opened, a file object is created and assigned to the provided identifier. Here, output_file is identifier.

The first argument is the file name to be opened, 'mynewfile.txt'. The second argument indicated file to be in writing format, however this can differ as per the operation.

Let’s Try

Navigation

1. What is Text Files?
2. Using Text Files
3. Why open File?
4. How to open Text files?
5. I/O Error
6. File saved in different directory
7. Absolute Path
8. Reading File
9. Writing File

Try Outs !!

In a new Python file window, enter the following lines,

```
file_name = input('Enter file name: ')
file = open(file_name, 'w')
file.close()
```

Save the file under the name `createfile.py` and run it. When the program requests a file name, give it any file name you wish with the extension `'.txt'`. Then, look in the folder in which the program resides to see if a new file with the file name that you entered exists. (This file will be empty—we will see how to write to a file next.)

Modify the open instruction above, changing `'w'` to `'r'` and rerun the program. When it requests a file name, enter the name of the file you just created. Run it a second time; this time give it the wrong file name. Observe the different results in each case.

Let’s Understand Through Example

Navigation

1. What is Text Files?
2. Using Text Files
3. Why open File?
4. How to open Text files?
5. I/O Error
6. File saved in different directory
7. Absolute Path
8. Reading File
9. Writing File

Example !!

```
user_input = input("Enter File You Want To Open: ")

input_file = open(user_input, ‘w’)

input_file.write("""
    Name: Ram Shakya
    Bank Account: 2265156331
    Balance: xxxxxxx
    """)

input_file = open(user_input, 'r')
print(input_file.read())
input_file.close()
```

Here, we can see a file is opened in a write format the previous file has been overwritten with the details that have been provided.

We Can Also Create Another Text File And Copy The Content From Existing.

Writing Into Text Files

Navigation

1. What is Text Files?
2. Using Text Files
3. Why open File?
4. How to open Text files?
5. I/O Error
6. File saved in different directory
7. Absolute Path
8. Reading File
9. Writing File
9.1 Writing into text files

- The **write method** is used to write string files

Text File
myfile.txt

```
Line One\n
Line Two\n
Line Three\n
```

```
empty_str = ''
input_file = open('myfile.txt','r')
output_file = open('myfile_copy.txt','w')

line = input_file.readline()

while line != empty_str:
    output_file.write(line)
    line = input_file.readline()

output_file.close()
```

Text File
myfile_copy.txt

```
line one
line two
line three
```

Visualization

Navigation

8. Reading File

9. Writing File

9.1 Writing into text files

Visualization !!

- **The code copies the contents of the input file**, '**myfile.txt**', line by line to the output file, '**myfile_copy.txt**'

- In contrast to print when writing to the screen, ***the write method does not add a newline character to the output string.*** Thus, a newline will be output only if it is part of the string being written. In this case, each line read contains a newline character.

Let's Try

Navigation

8. Reading File

9. Writing File

9.1 Writing into text files

Tryouts!!

Do the following in the Python shell:

```
input_file = open('myfile.txt','r')
output_file = open('newfile.txt','w')
line = input_file.readline()
output_file.write(line)
output_file.close()
```

Check if newfile.txt has been created

Do We Have To Close The File?

Closing Text Files

Navigation

1. What is Text Files?
2. Using Text Files
3. Why open File?
4. How to open Text files?
5. I/O Error
6. File saved in different directory
7. Absolute Path
8. Reading File
9. Writing File
10. Closing Text Files

- When the program has finished reading the file, it should be **closed** by class the **close method** on the file object:

```
input_file.close()
```

- Once closed, the file may be reopened (with reading starting at the beginning of the file) by the same or another program.
- Forgetting to close file can cause a host of issues. Changes may not be saved until the file closes and it can prevent removable media from being ejected.

String Processing

String Processing

Navigation

1. What is Text Files?
2. Using Text Files
3. Why open File?
4. How to open Text files?
5. I/O Error
6. File saved in different directory
7. Absolute Path
8. Reading File
9. Writing File
10. Closing Text Files
11. String Processing

- String processing refers to the various operations performed on strings, which include **searching, analysing, and updating information** in a text file or any other string-based data source.
- Previously, we discussed **str[k], for accessing individual characters**, **and len(str) for getting the length of a string.**
- We will revisit some sequence operations that apply to strings, and look at additional string specific methods that help us when processing files.

String Traversing

String Traversal

Navigation

8. Reading File

9. Writing File

10. Closing Text Files

11. String Processing

11.1 String Traversal

- We saw in an earlier lecture how any sequence can be traversed, including strings. This is usually done by the use of **for loop**.
- [**For instance**] If we want to read a line of text files and determine the count of blank characters contains then we can do following:

```
space = ' '
num_spaces = 0
line = input_file.readline()
for k in range(0,len(line)):
    if line(k) == space
    num_spaces = nm_spaces + 1
```

String Traversal without Index

Navigation

8. Reading File

9. Writing File

10. Closing Text Files

11. String Processing

11.1 String Traversal

11.2. String Traversal without index

- **The traversal can also be done more simply without the explicit use of index variable:**

```
for chr in line:
    if chr == space:
        num_spaces = nm_spaces + 1
```

- Given the ability to traverse string, each character can be individually "**looked at**" for various types of string processing.

String Applicable Sequence Operations

Navigation

8. Reading File

9. Writing File

10. Closing Text Files

11. String Processing

11.1 String Traversal

11.2. String Traversal without index

11.3. Sequence operators for string

- Because **strings** are **immutable, sequence modifying operations** are **not applicable** to strings. However , there are other applicable sequence operations.

Sequences Operations Applicable to Strings			
Length	`len(str)`	Membership	`'h' in s`
Select	`s[index_val]`	Concatenation	`s + w`
Slice	`s[start:end]`	Minimum Value	`min(s)`
Count	`s.count(char)`	Maximum Value	`max(s)`
Index	`s.index(char)`	Comparison	`s == w`

String Applicable Sequence Operations

Navigation

8. Reading File
9. Writing File
10. Closing Text Files
11. String Processing
11.1 String Traversal
11.2. String Traversal without index
11.3. String applicable sequence operations continued

- **Recall that as we saw with the lists**, The slice operator **s[start:end]** returns the substring starting from index start and up to, but not including, index end, while **s.index(chr)** returns the index of the first occurrence of chr in s.

- When applied to strings, **min and max** functions return the smallest and largest characters based on Unicode encoding. As a result, lowercase letters have larger Unicode values than uppercase letters.

We did access traverse each string but can we compare or imply conditions to each characters?

String and Conditions

Navigation

8. Reading File

9. Writing File

10. Closing Text Files

11. String Processing

11.1 String Traversal

11.2. String Traversal without index

11.3. Sequence operators for string

11.4. String and Conditions

- **There are times when the individual characters in a string (or substring) needed to be checked?**
- [**For instance**] to check whether a character is an appropriate denotation of musical note, we could do the following:

```
if char not in ('A', 'B', 'C', 'D', 'E', 'F', 'G'):
    print('Invalid musical note found')
```

Let's Try!!

Navigation

8. Reading File

9. Writing File

10. Closing Text Files

11. String Processing

11.1 String Traversal

11.2. String Traversal without index

11.3. Sequence operators for string

11.4. String and Conditions

Try Outs!!

- Since the **in** operator can also be applied to strings, we can also do the following.

```
if char not in 'ABCDEFG':
    print('Invalid musical note found')
```

- **We could take a similar approach for determining if a given character is lowercase or uppercase letter or digit character**

```
char in 'ABCDEFGHIJKLMNOPQRSTUVWXYZ' or \
char in 'abcdefghijklmnopqrstuvwxyz' or \
char in '0123456789'
```

Other Built in String Methods

Navigation

8. Reading File

9. Writing File

10. Closing Text Files

11. String Processing

11.1 String Traversal

11.2. String Traversal without index

11.3. Sequence operators for string

11.4. String and Conditions

11.5. Other built in string method

- Since checking for **uppercase/lowercase** and **digit characters** is common in programming, Python provides string methods **isalpha, isdigit, isupper, and islower** (among others)
- [**For instance**] to perform error checking on an entered credit card number could be done like so

```
if not credit_card.isdigit():
    print('Invalid card number')
```

- The method isdigit() return true when there is only integer in credit_card variable.

Click here https://docs.python.org/3/library/stdtypes.html#string-methods

Thank
You!